

UAV SIMULATOR

Phase 3 Guide

AI-DRIVEN DESIGN OPTIMIZATION

Automatically searching propeller, frame, and component designs against real requirements

For every developer on the team, regardless of background. No prior optimization or aerodynamics experience assumed.

Assumes you've already read the Foundations Guide, and completed Phase 1 and at least the CFD portion of Phase 2.

A numbering note, since this changed partway through planning: this phase was originally called "Phase 4" in the master project spec, with Hardware-in-the-Loop as "Phase 3." The team has since decided to build this design-optimization work before the hardware-in-the-loop work, so this document (and all future documents) calls this phase "Phase 3." Hardware-in-the-Loop is deferred and will be built later, at which point it will be referred to as Phase 4.

How to Use This Document

Phase 1 gave the drone correct rigid-body flight. Phase 2 made its aerodynamics, structure, and power system realistic, and validated them against real-world data. Phase 3 uses all of that as the foundation for something more ambitious: an AI system that automatically searches through different propeller designs, frame designs, and component layouts, and finds good ones for a given set of real-world requirements — for both quadcopters and fixed-wing aircraft.

This is a genuinely advanced topic — the technique it's built on, Multidisciplinary Design Optimization, is a real, established field with entire aerospace teams dedicated to it (NASA's own OpenMDAO tool is a well-known example) — so this document explains everything from scratch, the same way the Phase 2 guide did, with no assumption that anyone already knows what optimization, Blade Element Momentum Theory, or a Pareto front is.

This document has ten parts:

- Part 1 — What Phase 3 Actually Is: the goal, in plain language.
- Part 2 — Why We Can't Just Run CFD/FEA on Every Candidate Design: the computational wall, and the way around it.
- Part 3 — The Fast Physics Models: Blade Element Momentum Theory, lifting-line theory, and how they get calibrated using Phase 2's validated data.
- Part 4 — The Parametric Design Space: what the AI is actually allowed to change, and why we start here instead of full freeform design.
- Part 5 — Requirements, Objectives, and Constraints: turning "needs 30-minute endurance" into numbers a computer can search against.
- Part 6 — The Search Loop: how the AI actually explores the design space, and why there's usually more than one "best" answer.
- Part 7 — How This Plugs Into What We Already Built: headless execution and the data flow of one search iteration.
- Part 8 — The Build Plan: step-by-step tasks, in order.
- Part 9 — Team Roles for Phase 3.
- Part 10 — Glossary.

This document assumes Phase 2's CFD pipeline (and ideally FEA) is already validated. Phase 3 doesn't just benefit from that validated data — it structurally depends on it, since it's exactly what makes this phase's fast models trustworthy instead of generic textbook guesses.

Part 1 — What Phase 3 Actually Is

Every previous phase made the simulation itself more accurate. Phase 3 does something different: it uses that accurate simulation as a scoring tool, and wraps a search algorithm around it that tries many different drone designs, scores each one, and gradually homes in on good designs — the same way a person might try many different recipes, taste each one, and refine toward something better, except here "tasting" means running the design through calibrated physics models.

Phase	What It Produces
Phase 1	A correct physics engine for one fixed, given drone design
Phase 2	A physics engine whose aerodynamics, structure, and power are realistic and validated against real-world data, still for one fixed given design
Phase 3	A system that searches across many possible designs (not just one fixed design) and finds good ones for a stated set of requirements

Concretely: you give the system a requirement like "2 kilograms of payload, 30 minutes of flight time, stable in 15 meters-per-second wind," and it searches through combinations of propeller shape, frame arm length and layout, motor placement, and (for fixed-wing designs) wing dimensions, and returns a design — or, more realistically, a small set of different designs representing different trade-offs — that best satisfies what you asked for.

Part 2 — Why We Can't Just Run CFD/FEA on Every Candidate Design

This is the constraint that shapes everything else in this document, so it's worth stating plainly. A real search process needs to evaluate a very large number of candidate designs — easily thousands, often far more — to have any real chance of finding a good one, the same way you'd need to try far more than a handful of recipes to reliably find a great one rather than just an okay one.

Even with the fast GPU-based CFD from Phase 2, one case still takes somewhere from about 15 minutes to an hour. Multiply that by thousands of candidate designs, and you're looking at months to years of compute for a single search — completely impractical, and this isn't a limitation specific to our hardware; it would be true on almost any hardware available to a small team.

The way around this, used throughout real aerospace conceptual design work, is the same core idea Phase 2 already introduced for CFD and FEA, just applied one level up: replace the expensive, exact calculation with a fast, approximate one for the vast majority of evaluations, and only call the expensive, exact calculation occasionally, to check and calibrate the fast one.

The difference from Phase 2's pattern: there, the surrogate was fast because it interpolated between known flight conditions for one fixed design. Here, the fast model needs to handle entirely new, never-before-simulated designs, not just new flight conditions for a design we've already studied. This is why Phase 3 uses established analytical physics methods (Part 3) rather than a pure data-interpolation surrogate — those methods can extrapolate to new geometries in a way a lookup-style surrogate fundamentally can't.

Part 3 — The Fast Physics Models

3.1 Blade Element Momentum Theory (BEMT)

BEMT is a decades-old, well-established method for estimating a propeller's thrust and torque directly from its blade geometry, in milliseconds rather than minutes. The idea is genuinely intuitive once you see it: instead of simulating the whole blade at once, BEMT slices the blade into many small radial sections (imagine cutting a propeller blade into thin rings, from the center hub out to the tip), and treats each tiny section as its own miniature, straight airfoil moving through the air at its own local speed and angle — which depends on how far that section is from the hub, since the tip of a spinning blade moves much faster through the air than the part near the center.

Each section's tiny contribution to thrust and torque is calculated using simple, classical airfoil lift/drag relationships, combined with basic momentum theory (accounting for how the propeller accelerates air backward through its disk). Add up every section's contribution along the whole blade, and you get the propeller's total thrust and torque for that RPM and airspeed — all without ever simulating the actual moving air as a fluid.

BEMT's Known Weak Spot, and How We Fix It

BEMT's simplifying assumptions make it less accurate at low Reynolds numbers — the regime small UAV propellers often actually operate in, where air behaves slightly differently than the classical airfoil relationships BEMT is built on assume. Left uncorrected, BEMT can meaningfully mispredict small-propeller performance.

This is exactly why Phase 2's validated CFD data and the UIUC real-world measurements matter here, not just for Phase 2's own purposes: they become calibration data for BEMT. Concretely, you run BEMT and your validated CFD (or real UIUC data) on the same set of known propellers, measure the difference between them, and fit a correction — a relatively simple adjustment layered on top of BEMT's raw output — that brings it back in line with the real, trusted answers across the range of propeller sizes and speeds you actually care about.

3.2 Lifting-Line Theory — the Same Idea, for Wings

For the fixed-wing side of this project, lifting-line theory (a classical method developed by Ludwig Prandtl in the early 20th century, still standard in aircraft conceptual design today) plays the same role BEMT plays for propellers: it estimates a wing's lift and drag directly from its shape — span, chord, airfoil section, taper — without needing to simulate the surrounding air as a fluid. It works by representing the wing as a single line of varying-strength "bound vorticity" (a simplified stand-in for the swirling air the wing's lift actually creates) and calculating how lift is distributed along the wingspan from that.

Like BEMT, it has known assumptions and limitations — it works best for long, slender (high aspect ratio) wings at moderate angles of attack, and gets less accurate for very short, stubby wings or near stall. The same calibration approach applies: compare its predictions against your Phase 2 CFD results for representative wing shapes, and correct accordingly.

3.3 The Frame Aerodynamics Surrogate

For the drone's frame itself (as opposed to its propellers or wings), the most direct approach is to extend the exact GPR surrogate built in Phase 2 — but add the frame's own design parameters (arm length, arm angle, and so on, defined in Part 4) as extra input dimensions alongside the flight condition inputs it already has. This means the surrogate now answers "what's the drag/lift for this specific frame shape, at this specific flight angle and speed" instead of just "at this flight angle and speed," for one previously-fixed frame.

3.4 Calibration, Summarized

Across all three fast models, the same discipline applies: the fast model makes a quick, rough prediction; validated CFD (and where available, real physical measurements like the UIUC database) provides the trusted answer for a sample of cases; the difference between them is measured and corrected for. This calibration step is what turns "a fast textbook formula" into "a fast, trustworthy stand-in for our specific real-world-validated simulation" — skipping it would mean the search algorithm spends thousands of iterations optimizing against a model that might be confidently wrong.

Part 4 — The Parametric Design Space

4.1 Why Parametric Instead of Freeform

The original project vision described the AI freely adding material, removing material, and relocating screw holes on a 3D mesh — true topology optimization, where the design's entire shape is up for grabs. That's a genuinely difficult, active-research-level problem, and treating it as a starting point risks the whole phase stalling on its hardest possible version.

The practical starting point used throughout real aerospace conceptual design is a parametric design space instead: describe a design using a fixed list of adjustable numbers (a propeller's diameter and pitch, a frame arm's length and angle, and so on) rather than letting the shape itself be freely redrawn. The AI's job becomes choosing values for each of these numbers, not reshaping a mesh freely. Think of it like designing a car by choosing wheelbase, engine size, and seating capacity from adjustable sliders, rather than sculpting the entire car's shape from a lump of clay — far more tractable, while still covering a huge and genuinely useful range of possible designs. Full freeform topology optimization remains a legitimate future stretch goal once this parametric version is working well.

4.2 Quadcopter Frame Parameters (a Starting List)

- Arm length (distance from center to each motor)
- Arm layout angle (the "X" configuration spread, and whether it's symmetric)
- Arm cross-section thickness/shape (affects both structural strength and drag)
- Motor mount position along each arm
- Propeller diameter and pitch (either chosen from a catalog of known, already-validated propellers, or as its own separate custom design search using Part 3.1's BEMT model)
- Frame material choice (if more than one is being considered, e.g. different carbon fiber layup schedules)

4.3 Fixed-Wing Parameters (a Starting List)

- Wingspan and chord (which together determine aspect ratio — a major driver of lift/drag efficiency)
- Wing sweep angle
- Airfoil section choice (selected from known, well-studied airfoil shapes rather than designed freeform)
- Tail surface sizing (horizontal and vertical stabilizer dimensions)
- Propeller diameter and pitch, same as the quadcopter case

4.4 The Design Vector

All of a design's parameters, bundled together into one ordered list of numbers, form what's called a design vector — conceptually the same idea as Phase 1's `DroneState` bundling position, velocity, orientation, and angular velocity into one thing. Just as the RK4 integrator operates on a `DroneState` without caring what each individual number "means" internally, the search algorithm in Part 6 operates on a design vector the same way — proposing new candidate vectors, scoring them using Part 3's fast models, and refining its proposals based on those scores.

Part 5 — Requirements, Objectives, and Constraints

5.1 What MDO Is

Multidisciplinary Design Optimization (MDO) is the established engineering field this whole phase is really an application of — the practice of searching a design space where multiple different kinds of physics (aerodynamics, structure, weight, power) all interact and all matter simultaneously, rather than optimizing one discipline in isolation. It's a real, mature field, not something specific to this project — NASA's OpenMDAO is a well-known open-source framework built specifically for this purpose.

5.2 Turning Requirements Into Numbers

A requirement stated in plain English ("needs to carry 2 kilograms, fly for 30 minutes, remain stable in 15 meter-per-second wind") has to be translated into explicit, computable numbers before any search algorithm can use it: a minimum payload capacity, a minimum flight endurance calculated from battery capacity and estimated power draw, a maximum allowable tilt or instability metric at a given wind speed. This translation step is a real design decision in its own right — done carelessly, the algorithm will happily satisfy the literal numbers you gave it while producing a design nobody actually wants.

5.3 Objectives vs. Constraints

It's worth being precise about a distinction that's easy to blur: a constraint is a hard pass/fail requirement (the design must have a structural safety factor above a certain minimum, or it's simply rejected, no matter how well it scores elsewhere), while an objective is something you want to do as well as possible on, with more being better rather than pass/fail (minimize drag, minimize mass, maximize thrust-to-weight ratio). Structural integrity and basic safety margins should almost always be hard constraints, not soft objectives — you don't want an optimizer quietly trading away safety margin in exchange for a slightly better drag number.

5.4 The Score (Reward) Function

Everything above feeds into one function the search algorithm actually uses to compare candidate designs: reject anything that fails a hard constraint outright, and for everything that passes, combine the objectives (drag, mass, thrust-to-weight, and whatever else matters for this project) into a single score, typically as a weighted combination reflecting which objectives matter more for a given use case. This score is what the search algorithm in Part 6 is actually trying to make as good as possible.

Part 6 — The Search Loop

6.1 How the Search Actually Works, Step by Step

1. Propose a candidate design — a specific set of values for every parameter in the design vector (Part 4.4).
2. Score it using the fast, calibrated models from Part 3, checking every hard constraint from Part 5.3 and calculating the combined objective score for anything that passes.
3. Use that score to inform the next candidate design proposed — the exact mechanism for this depends on which search method is used (Part 6.3).
4. Repeat many thousands of times, gradually converging on designs that score well.

6.2 Why There's Usually More Than One "Best" Design — the Pareto Front

A design optimized purely for flight endurance and one optimized purely for payload capacity are usually different designs — pushing further on one typically costs you something on the other. Rather than forcing the search to collapse this trade-off into one single number (which hides real decisions a human should get to make), real MDO tools instead produce a Pareto front: a set of designs where none of them can be improved on one objective without making another objective worse. Every design on that front is a legitimate "best" answer, just for a different balance of priorities — the search's job is to map out that whole set of good trade-offs, and a human (or the specific mission requirement) picks which point on it actually matters most for a given use case.

6.3 Choosing a Search Method — an Open Decision Worth Flagging Honestly

The original project vision specifically called for reinforcement learning (RL), where an AI agent learns through repeated trial and reward. RL is a completely valid choice here, but it's worth being upfront: for a moderate-sized parametric design space like the one in Part 4 (tens of numbers, not thousands), simpler optimization methods — genetic algorithms (which evolve a population of candidate designs through simulated selection and mutation) or Bayesian optimization (which is philosophically very similar to the active-learning approach already used for Phase 2's CFD sweep, just searching design parameters instead of flight conditions) — are often more sample-efficient and considerably easier to get working reliably as a first version.

This is a genuine open decision for the team to make deliberately, not something to default into. A reasonable path: prototype with Bayesian optimization or a genetic algorithm first, since both are simpler to implement correctly and need fewer total design evaluations to find good results, and treat full reinforcement learning as a later upgrade once the overall pipeline (fast models, constraints, scoring, Pareto front output) is proven working end to end.

6.4 The Final Validation Pass

Once the search converges on one or a small handful of promising designs (or a full Pareto front of them), those specific designs — and only those, not every candidate considered along the way — get one final, expensive validation pass: real CFD and FEA, run properly, the same way Phase 2 validates any single design. This is what turns "the fast calibrated models think this is good" into "we've actually confirmed this with the same rigor as everything else in this project," before anyone treats a design as genuinely finished.

Part 7 — How This Plugs Into What We Already Built

7.1 Headless Execution

Running thousands of design evaluations doesn't need Unreal Engine's graphics at all — there's no need to render a single frame while the search is running. This phase strips the graphics away entirely, running the Rust physics core (and the fast models from Part 3) as a plain, high-speed command-line loop. This is exactly why Phase 1's architecture decision to keep Rust as a self-contained library, callable independently of Unreal Engine, matters here: the same physics core built for the live simulator can run headless for this phase with no changes needed to that core itself.

7.2 The Data Flow of One Search Iteration

1. The search algorithm (Part 6) proposes a candidate design vector (Part 4.4).

2. The candidate is checked against hard constraints (Part 5.3); if it fails, it's rejected immediately with no further computation spent on it.
3. If it passes, the fast, calibrated models from Part 3 (BEMT, lifting-line theory, the extended frame surrogate) estimate its aerodynamic and structural performance.
4. These estimates feed into the combined objective score (Part 5.4).
5. The score is returned to the search algorithm, which uses it to propose the next candidate.
6. After the search converges, the small number of final candidates go through the full validation pass (Part 6.4) using real CFD/FEA, exactly as in Phase 2.

Part 8 — The Build Plan: Step-by-Step Tasks, In Order

This is the order Phase 3 should actually be built in. It assumes Phase 2's CFD pipeline (and the UIUC-based validation from that phase) is already working.

Task 1 — Implement BEMT

1. Build a basic Blade Element Momentum Theory calculator that takes blade geometry, RPM, and airspeed, and returns thrust and torque.
2. Test it first against a UIUC propeller with published real measurements, before comparing it to anything from your own CFD.

Task 2 — Calibrate BEMT Against Validated Data

1. Run BEMT and your validated Phase 2 CFD (or UIUC real data) on the same set of known propellers across a range of RPM and airspeed.
2. Measure the difference between BEMT's raw output and the trusted values.
3. Fit a correction that brings BEMT's predictions in line with the trusted data across the range you care about.

Task 3 — Implement and Calibrate Lifting-Line Theory (If Building Fixed-Wing Designs)

1. Build a basic lifting-line calculator for a given wing shape.
2. Calibrate it the same way as Task 2, using Phase 2 CFD results for representative wing shapes.

Task 4 — Extend the Frame Surrogate With Design Parameters

1. Add frame design parameters (Part 4.2) as new input dimensions to the existing Phase 2 GPR surrogate.
2. This will likely require additional CFD cases beyond Phase 2's original flight-condition-only sweep, covering a sample of different frame shapes as well — plan for this as new offline compute work, not something the existing dataset already covers.

Task 5 — Define the Full Design Vector

1. Finalize the exact parameter list from Part 4.2 and 4.3 for whichever aircraft type (quadcopter, fixed-wing, or both) this phase targets first.
2. Define sensible minimum/maximum ranges for every parameter.

Task 6 — Translate Requirements Into Objectives and Constraints

1. Take a real, specific example requirement and translate it into explicit target numbers and hard constraints, following Part 5.2 and 5.3.
2. Define the combined objective score function from Part 5.4.

Task 7 — Build the Headless Evaluation Loop

1. Strip Unreal Engine out of the loop entirely, running the Rust physics core plus the new fast models as a standalone command-line program.
2. Confirm it can take one design vector and return one score, correctly rejecting designs that fail hard constraints.

Task 8 — Implement the Search Algorithm

1. Start with Bayesian optimization or a genetic algorithm (Part 6.3) rather than full reinforcement learning, for a simpler first working version.
2. Confirm the search actually improves its proposed designs' scores over successive iterations on a small test run before scaling up.

Task 9 — Run a Full Search and Produce a Pareto Front

1. Run the search to completion against a real requirement from Task 6.
2. Produce the set of Pareto-optimal designs (Part 6.2) rather than collapsing the output to one single design.

Task 10 — Final Validation Pass

1. Take the small number of final candidate designs from Task 9 and run full, real CFD/FEA on each, exactly as in Phase 2.
2. Confirm the fast models' predictions were actually close to these real results — if they weren't, revisit the calibration in Task 2/3 before trusting future search runs.

Testing and debugging procedures for each of these tasks follow the same pattern established in Phase 1 and Phase 2: per-task quick checks, then a full master test list before considering a task done. A dedicated Phase 3 Testing & Debugging document can be built once ownership of these tasks is assigned.

Part 9 — Team Roles for Phase 3

Like Phase 2, this is a suggested starting split rather than a fixed assignment.

Suggested Role	Owns	Builds On
Fast Models & Calibration Owner	Tasks 1–4: BEMT, lifting-line theory, calibration against Phase 2 data, extended frame surrogate	Whoever owned Phase 2's CFD/surrogate work is the most natural fit
Requirements & Scoring Owner	Tasks 5–6: design vector definition, objective/constraint translation	Needs close coordination with whoever defines real target requirements for the company
Search & Optimization Owner	Tasks 7–9: headless loop, search algorithm, Pareto front output	New role for this phase — optimization/ML background is most relevant here
Validation Owner	Task 10: final CFD/FEA validation pass	Same skill set as Phase 2's CFD/FEA validation work

Part 10 — Glossary (Phase 3 Terms)

New terms introduced in this document. For Phase 1 and Phase 2 terms, see their respective glossaries.

Term	Plain-English Meaning
Aspect ratio	A wing's span divided by its chord; a major driver of how efficiently a wing generates lift versus drag.
Bayesian optimization	A search method that uses a probabilistic model (similar in spirit to the GPR surrogate from Phase 2) to decide which candidate design to try next, balancing exploring the unknown against refining known-good areas.
BEMT (Blade Element Momentum Theory)	A fast, classical method for estimating propeller thrust and torque by dividing the blade into small sections and analyzing each as a mini-airfoil.
Calibration (in this context)	Correcting a fast approximate model's predictions using differences measured against trusted, validated data.
Constraint	A hard pass/fail requirement a design must satisfy, regardless of how well it scores on anything else.
Design vector	The full set of a design's adjustable parameters, bundled together as one ordered list of numbers.
Genetic algorithm	A search method that evolves a population of candidate designs over generations, using simulated selection, mutation, and combination.
Headless execution	Running a program with no graphical rendering at all, for speed — used here to run thousands of design evaluations without ever rendering a single frame.
Lifting-line theory	A classical method (developed by Ludwig Prandtl) for estimating a wing's lift and drag distribution from its shape, without simulating the surrounding air as a fluid.
MDO (Multidisciplinary Design Optimization)	The established engineering field of searching a design space where multiple kinds of physics all interact and matter simultaneously.
Objective	A quantity the search tries to do as well as possible on (like minimizing drag), as opposed to a hard pass/fail constraint.
Parametric design space	A design space described by a fixed list of adjustable numbers (like arm length or wingspan), rather than a fully freeform shape.
Pareto front	The set of designs where none can be improved on one objective without making another objective worse — the set of legitimately "best" trade-off answers, rather than one single best design.
Reynolds number	A dimensionless number describing the relationship between a fluid's inertia and its viscosity for a given size and speed; small UAV propellers often operate at low Reynolds numbers, a regime where some classical aerodynamic assumptions become less accurate.
Topology optimization	A design approach where a shape's entire material layout is freely added, removed, or reshaped, rather than adjusted through a fixed set of parameters — the hardest, most open-ended version of design optimization.